Day 149 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 3 reference

Watch: ARIMA & SARIMA Made Easy: Forecasting for Beginners (Prof. Ryan Ahmed) | Coding ARMA Model: Time Series Talk (ritvikmath)

`day149_arima_from_scratch.py`, segment by segment, in the order `main()` uses things. The partial autocorrelation, the seasonal ARMA polynomials, the error recursion, the Levenberg-Marquardt solver, the CSS fit, the differencing polynomial, the forecasts, the psi weights and the Ljung-Box statistic are new, and are copied unchanged into today's exercise file. Days 147 and 148's parts are imported from their exercise files. scipy and statsmodels are used only as references to check against. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: w = differenced series · e = errors · B = lag · a(B), b(B) = AR and MA polynomials · A(B) = a(B) times the differences · ψ = psi weights

Segment 0 — Setup

Imports and constants

CODE

```
from scipy.optimize import least_squares
from scipy.signal import lfilter
from scipy.stats import chi2
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import pacf as sm_pacf
warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day147_exercise as d147
    import day148_exercise as d148
AIRLINE = (0, 1, 0, 1, 12)
```

- `from scipy.signal import lfilter` — scipy's recursive filter, used only to check our error recursion.
- `from statsmodels.tsa.statespace.sarimax import SARIMAX` — the library's seasonal ARIMA, fitted by exact likelihood; the reference.
- `import day148_exercise as d148` — yesterday's parts: d148.acf and d148.difference.
- `AIRLINE = (0, 1, 0, 1, 12)` — a tuple (p, q, P, Q, s): one constant names the model used throughout.

Segment 1 — The parts

pacf: Durbin-Levinson

MATH

$$a = (r_k - \Sigma\, \phi_j\, r_{k-j}) \,/\, (1 - \Sigma\, \phi_j\, r_j), \quad \phi_j \leftarrow \phi_j - a\, \phi_{k-j}$$

CODE

```
def pacf(x, nlags):
    r = d148.acf(x, nlags); out = [1.0]; phi = np.zeros(0)
    for k in range(1, nlags + 1):
        a = (r[k] - phi @ r[k - 1:0:-1]) / (1 - phi @ r[1:k])
        phi = np.r_[phi - a * phi[::-1], a]; out.append(a)
    return np.array(out)
```

- `phi = np.zeros(0)` — an empty array: the AR fit of order 0 has no coefficients.
- `r[k - 1:0:-1]` — a backwards slice: r[k-1], r[k-2], ..., r[1] (the stop index 0 is not included).
- `phi @ r[1:k]` — @ on two 1-D arrays is a dot product; on empty arrays it gives 0.0.
- `np.r_[phi - a * phi[::-1], a]` — the updated coefficients with the new one appended at the end.

sarima_polys and css_residuals

MATH

$$e_t = a(B)\, w_t - \Sigma_j\, b_j\, e_{t-j}, \quad \textit{errors before the start} = 0$$

CODE

```
def sarima_polys(params, order):
    p, q, P, Q, s = order
    phi, theta = params[:p], params[p:p + q]
    PHI, THETA = params[p + q:p + q + P], params[p + q + P:p + q + P + Q]
    sa = np.zeros(s * P + 1); sa[0] = 1; sa[s::s] = -PHI
    sb = np.zeros(s * Q + 1); sb[0] = 1; sb[s::s] = THETA
    return np.convolve(np.r_[1.0, -phi], sa), np.convolve(np.r_[1.0, theta], sb)
def css_residuals(w, a, b, start=None):
    r = len(a) - 1 if start is None else start
    u = np.convolve(w, a)[r:len(w)]
    q = len(b) - 1; lags = np.flatnonzero(b[1:]) + 1; bl = b[lags]
    e = np.zeros(q + len(u))
    for t in range(len(u)):
        e[q + t] = u[t] - bl @ e[q + t - lags]
    return e[q:]
```

- `p, q, P, Q, s = order` — tuple unpacking: five names from one tuple.
- `sa[s::s] = -PHI` — a step slice: positions s, 2s, ...; assigning an empty array to an empty slice does nothing.
- `np.convolve(np.r_[1.0, -phi], sa)` — multiplying two polynomials is convolving their coefficient lists.
- `np.flatnonzero(b[1:]) + 1` — the lags whose MA coefficient is not zero (1, 12 and 13 for the airline model).
- `e[q + t - lags]` — fancy indexing with an array: the earlier errors at exactly those lags.
- `return e[q:]` — the q leading zeros were only padding, so they are dropped.

levenberg_marquardt and fit_css

MATH

$$step = -(J^T J + \lambda\, diag\, J^T J)^{-1}\, J^T e; \quad \lambda \div 10 \text{ after a success, } \times 10 \text{ after a failure}$$

CODE

```
def levenberg_marquardt(fun, x0, iters=200, tol=1e-12):
    x = np.array(x0, float); r = fun(x); ssr = r @ r; lam = 1e-3
    for _ in range(iters):
        J = np.empty((len(r), len(x)))
        for j in range(len(x)):
            dx = np.zeros(len(x)); dx[j] = 1e-6 * max(1.0, abs(x[j]))
            J[:, j] = (fun(x + dx) - r) / dx[j]
        g, H = J.T @ r, J.T @ J
        while True:
            step = np.linalg.solve(H + lam * np.diag(np.diag(H) + 1e-12), -g)
            r_new = fun(x + step); ssr_new = r_new @ r_new
            if ssr_new < ssr:
                break
            lam *= 10
            if lam > 1e10:
                return x, ssr
        x, r, lam, gain, ssr = x + step, r_new, lam / 10, ssr - ssr_new, ssr_new
        if gain < tol * ssr:
            break
    return x, ssr
def fit_css(w, order, start=None):
    def res(th):
        return css_residuals(w, *sarima_polys(th, order), start=start)
    x, ssr = levenberg_marquardt(res, np.full(sum(order[:4]), 0.1))
    m = len(res(x))
    return x, ssr / m, m
```

- `for _ in range(iters):` — _ is a name for a loop counter that is never used.
- `J[:, j] = (fun(x + dx) - r) / dx[j]` — one column of the Jacobian by a forward difference.
- `np.linalg.solve(H + lam * np.diag(np.diag(H) + 1e-12), -g)` — np.diag of a matrix gives its diagonal; of a vector, a diagonal matrix.
- `while True: ... break` — an open loop, left by break when a step lowers the sum of squares.
- `x, r, lam, gain, ssr = ...` — five names updated at once; the right side is computed fully before any is assigned.
- `def res(th):` — a function inside fit_css; it remembers w, order and start (a closure).
- `sum(order[:4])` — Python's sum of the first four orders: the number of coefficients.

full_ar, forecast and psi_weights

MATH

$$A(B) = a(B)(1 - B)(1 - B^{12}); \quad \psi_j = b_j - \Sigma_{i=1..j} A_i \, \psi_{j-i}$$

CODE

```
def full_ar(a, d=1, D=1, s=12):
    for _ in range(d):
        a = np.convolve(a, [1.0, -1.0])
    for _ in range(D):
        a = np.convolve(a, np.r_[1.0, np.zeros(s - 1), -1.0])
    return a
def forecast(y, A, b, e, h):
    n = len(y); x = np.r_[y, np.zeros(h)]
    E = np.r_[np.zeros(n - len(e)), e, np.zeros(h)]
    for t in range(n, n + h):
        x[t] = -A[1:] @ x[t - 1:t - len(A):-1] + b[1:] @ E[t - 1:t - len(b):-1]
    return x[n:]
def psi_weights(A, b, h):
    p = np.zeros(h)
    for j in range(h):
        k = min(j, len(A) - 1)
        p[j] = (b[j] if j < len(b) else 0.0) - A[1:k + 1] @ p[j - k:j][::-1]
    return p
```

- `np.r_[1.0, np.zeros(s - 1), -1.0]` — the coefficients of $1 - B^{12}$: a 1, eleven zeros, a –1.
- `x[t - 1:t - len(A):-1]` — the last len(A) – 1 values, newest first, to pair with A[1], A[2], ...
- `np.r_[np.zeros(n - len(e)), e, np.zeros(h)]` — the errors padded to line up with y: zeros before they start and for the future.
- `b[j] if j < len(b) else 0.0` — a conditional expression: b has no coefficient beyond its length.
- `p[j - k:j][::-1]` — the k previous weights, newest first.

ljung_box

MATH

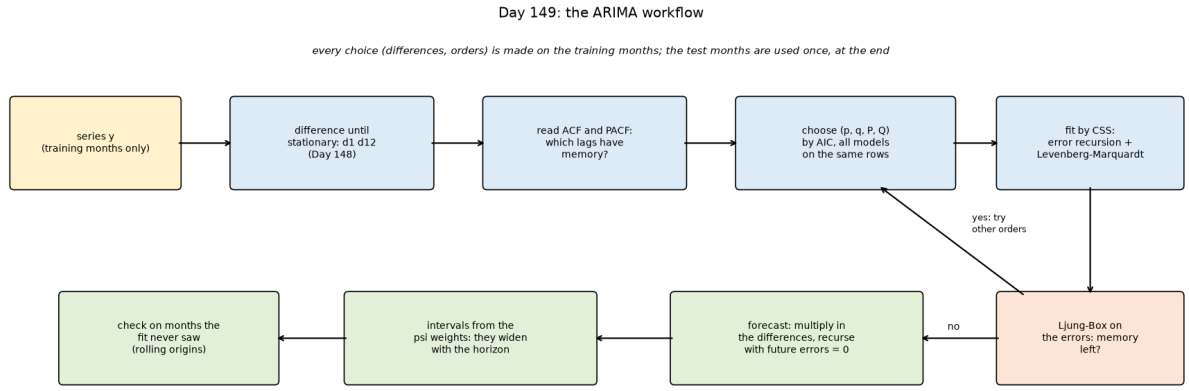
$$Q = n(n+2) \, \Sigma_{k=1..h} \, r_k^{\,2} / (n - k)$$

CODE

```
def ljung_box(e, lags, fitted=0):
    r = d148.acf(e, lags); n = len(e)
    return n * (n + 2) * np.sum(r[1:] ** 2 / (n - np.arange(1, lags + 1))), lags - fitted
```

- `n - np.arange(1, lags + 1)` — an array n–1, n–2, ...: each lag's own divisor, in one operation.
- `return n * (n + 2) * ..., lags - fitted` — two values returned as a tuple, unpacked by the caller as Q, df = ...

SYSTEM DIAGRAM



The ARIMA workflow.

Segment 2 — The experiments

simulate_arma: series with known coefficients

CODE

```
def simulate_arma(phi, theta, n=500, seed=0, burn=100):
    e = np.random.RandomState(seed).standard_normal(n + burn); x = np.zeros(n + burn)
    for t in range(max(len(phi), len(theta)), n + burn):
        x[t] = (sum(f * x[t - i - 1] for i, f in enumerate(phi)) + e[t]
                + sum(g * e[t - j - 1] for j, g in enumerate(theta)))
    return x[burn:]
```

- `enumerate(phi)` — pairs (0, phi_1), (1, phi_2), ...: the index gives the lag.
- `sum(f * x[t - i - 1] for i, f in enumerate(phi))` — a generator inside Python's sum; for an empty list the sum is 0.
- `x[burn:]` — the first 100 steps dropped, so the start-up from zeros does not show.

demo_fit: three references

CODE

```
ref = lfilter([1.0], b, np.convolve(x, a)[1:len(x)])
print(f"error recursion against scipy's lfilter, at the true coefficients: largest difference
    {np.max(np.abs(css_residuals(x, a, b) - ref)):.1e}")
t0 = time.perf_counter(); th, s2, m = fit_css(x, order); dt = time.perf_counter() - t0
ls = least_squares(lambda v: css_residuals(x, *sarima_polys(v, order)), [0.1, 0.1], xtol=1e-15,
    ftol=1e-15, gtol=1e-15).x
ml = ARIMA(x, order=(1, 0, 1), trend="n").fit().params
```

- `lfilter([1.0], b, u)` — solves b(B) e = u for e, the same recursion as css_residuals.
- `least_squares(lambda v: ..., [0.1, 0.1], xtol=1e-15, ...)` — a lambda is a one-line function; the tolerances make scipy run to full precision.
- `trend="n"` — no constant, to match our model.

demo_choose: the AIC grid

CODE

```
for p in range(3):
    for q in range(3):
        for P in range(2):
            for Q in range(2):
                _, s2, m = fit_css(w, (p, q, P, Q, 12), start=14)
                grid.append((m * np.log(s2) + 2 * (p + q + P + Q), (p, q, P, Q), s2))
grid.sort(key=lambda g: g[0])
```

- `start=14` — every model starts its errors at row 14, so all sums cover the same rows.
- `grid.append((aic, (p, q, P, Q), s2))` — a list of tuples, one per model.
- `grid.sort(key=lambda g: g[0])` — sorts by the first item only: AIC.

demo_forecast: checks against statsmodels

CODE

```
kal = SARIMAX(yc[:n], order=(0, 1, 1), seasonal_order=(0, 1, 1, 12)).filter(np.r_[th, s2])
print(f"       statsmodels' forecast with our coefficients: {np.max(np.abs(kal.forecast(h) -
    f)):.1e}; "
      f"interval widths: {np.max(np.abs(kal.get_forecast(h).se_mean - sd)):.1e}")
```

- `.filter(np.r_[th, s2])` — runs statsmodels' Kalman filter with our coefficients and error variance, without fitting.
- `kal.get_forecast(h).se_mean` — the forecast standard errors, to compare with our psi-weight ones.

demo_css_vs_mle: a default argument as a snapshot

CODE

```
for burn in (0, 13, 26, 39):
    def res(v, burn=burn):
        return css_residuals(w, *sarima_polys(v, AIRLINE))[burn:]
    v = levenberg_marquardt(res, [0.1, 0.1])[0]
    if burn == 0:
        v0 = v
```

- `def res(v, burn=burn):` — the default is evaluated when the function is defined, so each loop's res keeps its own burn.
- `css_residuals(...)[burn:]` — the first burn errors left out of the sum.
- `if burn == 0: v0 = v` — keeps the plain fit for the comparison printed afterwards.

demo_rolling: 97 forecasts

CODE

```
for o in origins:
    w = d148.difference(d148.difference(yc[:o], 12)); th, s2, _ = fit_css(w, AIRLINE); a, b =
        sarima_polys(th, AIRLINE)
    A = full_ar(a); f = forecast(yc[:o], A, b, css_residuals(w, a, b), 24); act = yc[o:o + 24]
    E["airline, ours (CSS)"].append(f - act); cover.append(np.abs(f - act) < 1.96 * np.sqrt(s2 *
        np.cumsum(psi_weights(A, b, 24) ** 2)))
    E["same month last year"].append(yc[o - 12:o][np.arange(24) % 12] - act)
```

- `fit_css(w, AIRLINE)` — the model is refitted at every origin, on the data before it only.
- `cover.append(np.abs(f - act) < 1.96 * ...)` — a True/False array per origin: was each month inside the 95% interval?
- `np.arange(24) % 12` — 0, 1, ..., 11, 0, 1, ...: last year's months repeated for two years.

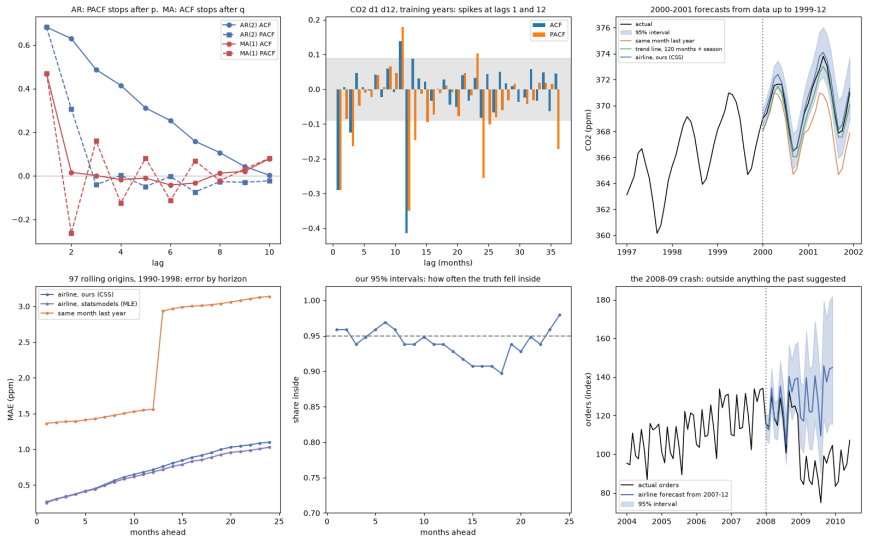
demo_orders: percentage errors

CODE

```
E["airline (CSS)"].append(np.abs(np.exp(f - act) - 1)); E["same month last
    year"].append(np.abs(np.exp(le[o - 12:o] - act) - 1))
E["last value"].append(np.abs(np.exp(le[o - 1] - act) - 1))
```

- `np.exp(f - act) - 1` — on a log scale a difference of logs is a ratio: exp(log f – log y) – 1 is the percentage error.
- `le[o - 12:o] - act` — the 12 months before the origin, compared month by month with the 12 being forecast.
- `np.exp(le[o - 1] - act)` — le[o - 1] is a single number, so it is broadcast against all 12 actual values.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.convolve(a, b)` — multiply two polynomials given by their coefficients.
- `x[k - 1:0:-1]` — a backwards slice that stops before index 0.
- `e[q + t - lags]` — fancy indexing: several earlier values at once.
- `lambda v: expr` — a one-line function without a name.
- `def f(v, burn=burn):` — a default argument fixes a loop variable's current value.
- `{k: f(v) for k, v in d.items()}` — a dictionary comprehension.
- `SARIMAX(...).filter(params)` — statsmodels: run the model with given coefficients, no fitting.

End of Day 149 syntax reference. For the experiments and what they showed, see the Day 149 complete guide.